



Course Code : SWE310

Course Name : Programming Elective II (2)

Lecturer : Teo Bee Guan

Academic Session : 2026/04

Assessment Title : Assignment

Submission Due Date : 31 May 2025 (23.59 PM)

Prepared by :	Student ID	Student Name
	SWE2309434	Laurenz William

Feedback from Lecturer:	
	Mark:

Own Work Declaration

I/We acknowledge that my/our work will be examined for plagiarism or any other form of academic misconduct, and that the digital copy of the work may be retained for future comparisons.

I/We confirm that all sources cited in this work have been correctly acknowledged and that I/we fully understand the serious consequences of any intentional or unintentional academic misconduct.

In addition, I/we affirm that this submission does not contain any materials generated by AI tools, including direct copying and pasting of text or paraphrasing. This work is my/our original creation, and it has not been based on any work of other students (past or present), nor has it been previously submitted to any other course or institution.

Date: 31 May 2026

Signature:

A handwritten signature in black ink, appearing to read 'Mulliam'.

Table of Content

1. Application Overview & Justification of Chosen Topic.....	1
2. ERD & Explanation of The Database Schema.....	1
3. API Endpoint Summary.....	3
3.1 Student Table.....	3
3.2 Test Table.....	4
3.3 Student Evaluation Table.....	6
4. UI Design.....	6
4.1 DashBoard Page.....	6
4.2 Course Page.....	7
4.3 Student Page.....	9
4.4 Student Evaluation Page.....	12

1. Application Overview & Justification of Chosen Topic

I chose the topic "Student Grade Tracker" in developing a fullstack application consisting of front end and back end. The reason is that lecturers/teachers do not have a detailed picture of and it is time-consuming if calculating grades, moving data and compiling reports manually. By creating a website called 'GradeTrac', lecturers/teachers can monitor student academic performance quickly and accurately, can help in inputting grades easily and monitor the grades obtained by students in the lesson. Additionally, can see the evaluation and feedback given by students. This website will be created with a combination of design and color that is eye friendly consisting of 4 different pages. There is a dashboard page, a course page, a student page, and a student evaluation page, each of which displays different data.

2. ERD & Explanation of The Database Schema

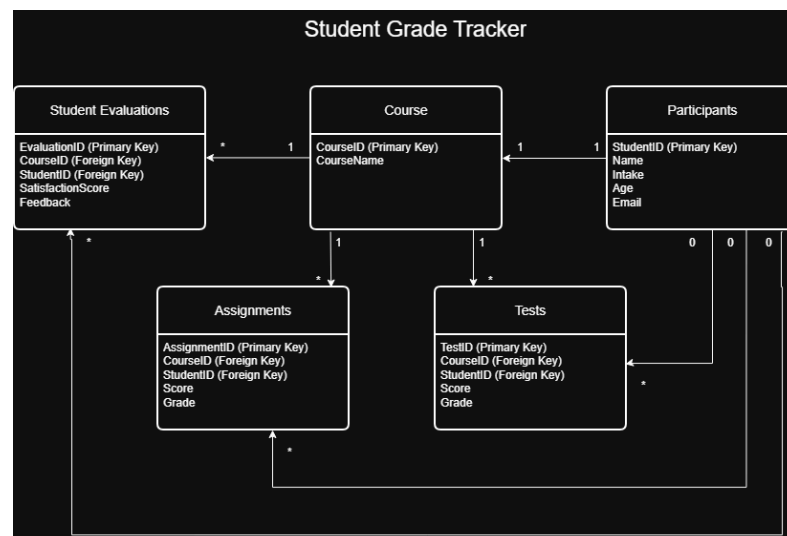


Figure 2.1 Entity Relationship Diagram

Zero to Many (0..*) means it is possible to have no data at all (zero), and if there is, the amount can be large (infinite). Example: Participants can have zero feedback but also can also have more than one feedback. One to Many (1..*) means requires the presence of at least one data item in the transaction table or child table for the relationship to be considered valid. Example: Inside the course should have at least 1 assignment and can have more than 1

assignment. Participants table used to store the identity profile of students registered in the tracker system. The course table maintains a list of courses available within the institution. Assignment table to recording student assignment performance scores in certain courses. Tests table used to record the results of formal exams or periodic tests taken by students. The Student Evaluations table maintains satisfaction rate and written reviews from students regarding the quality of the courses they complete. Each of the table contain a primary key.

```
CREATE TABLE StudentEvaluations (
    EvaluationID INT IDENTITY(1,1) PRIMARY KEY,
    CourseID VARCHAR(20) NOT NULL,
    StudentID VARCHAR(20) NOT NULL,
    SatisfactionScore INT CHECK (SatisfactionScore BETWEEN 1 AND 5),
    Feedback VARCHAR(MAX),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID) ON DELETE CASCADE,
    FOREIGN KEY (StudentID) REFERENCES Participants(StudentID) ON DELETE CASCADE
);
GO
```

Figure 2.2 StudentEvaluation Table

All the foreign keys at (Assignments, Test, StudentEvaluations) tables are configured with the **ON DELETE CASCADE** clause. We can take a look at one of the examples in the StudentEvaluations table. The purpose is If any student or course data is deleted from the master table, all evaluation text history related to that data will automatically be permanently deleted by the system. This is implemented to ensure data consistency.

3. API Endpoint Summary

GET /api/{entity} : Retrieves all data or a list from that entity.

GET /api/{entity}/{id} : Retrieves a specific data based on its ID.

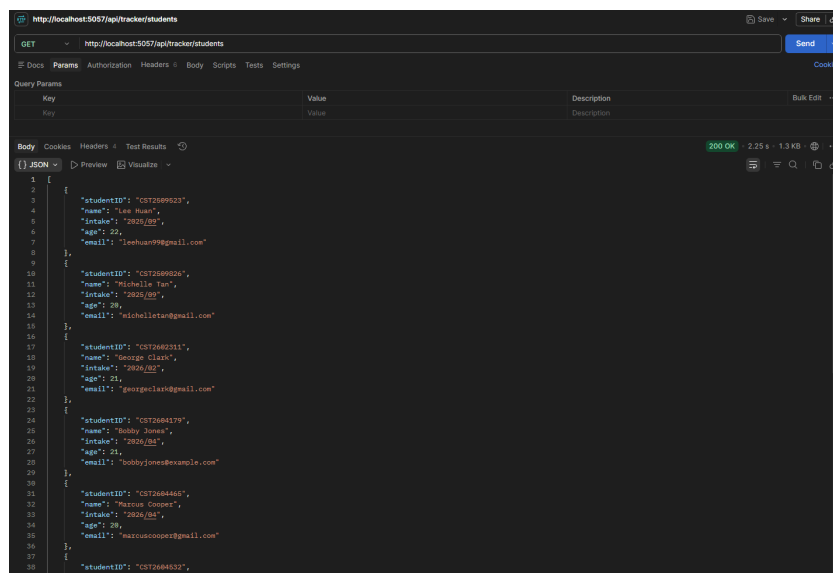
POST /api/{entity} : Create or add new data.

PUT /api/{entity}/{id} : Change or update existing data as a whole based on its ID.

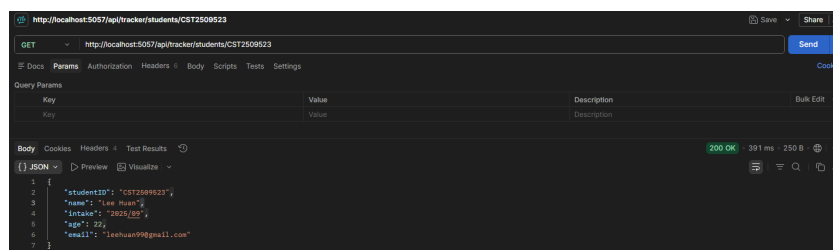
DELETE /api/{entity}/{id} : Deletes specific data based on its ID.

3.1 Student Table

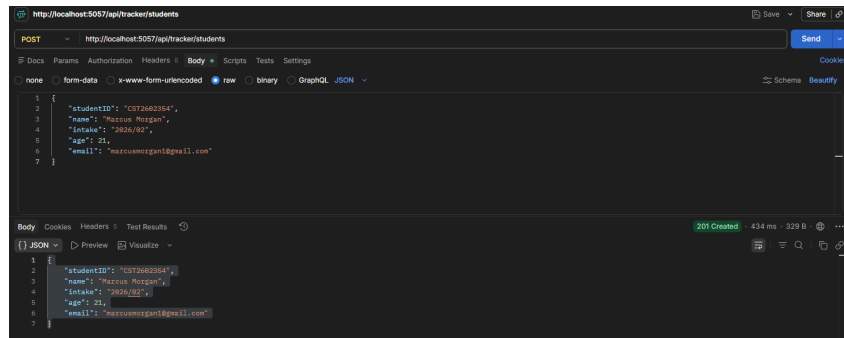
GET : <http://localhost:5057/api/tracker/students>



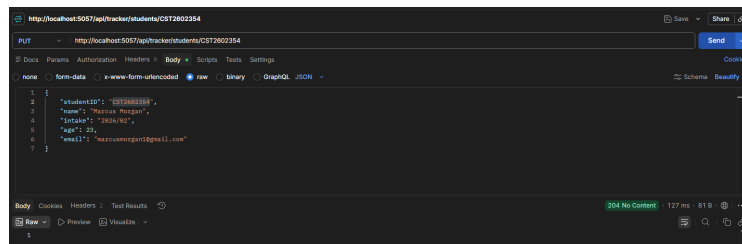
GET : <http://localhost:5057/api/tracker/students/CST2509523>



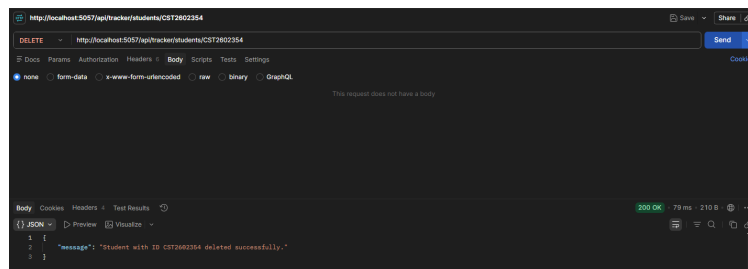
POST : <http://localhost:5057/api/tracker/students>



PUT : http://localhost:5057/api/tracker/students/CST2602354

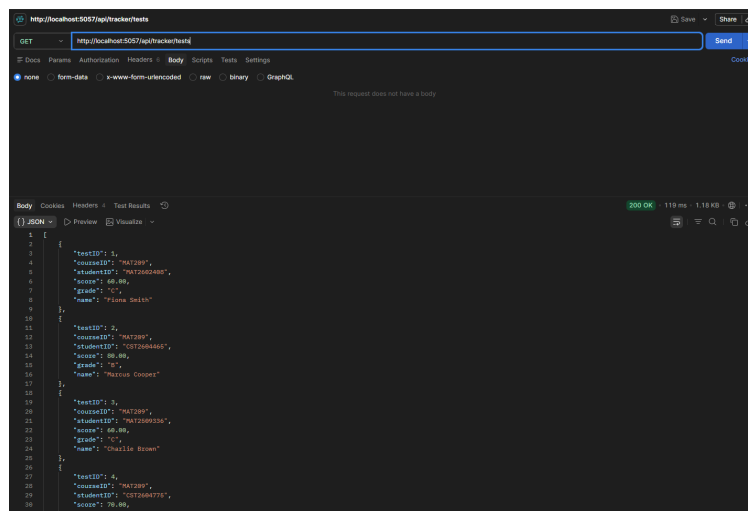


DELETE : http://localhost:5057/api/tracker/students/CST2602354



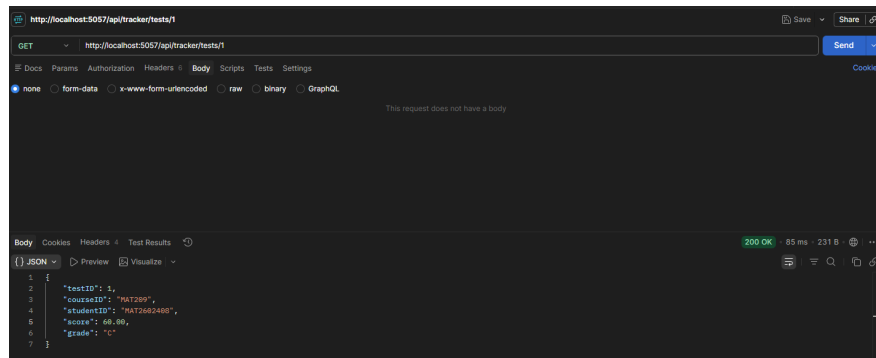
3.2 Test Table

GET : http://localhost:5057/api/tracker/tests

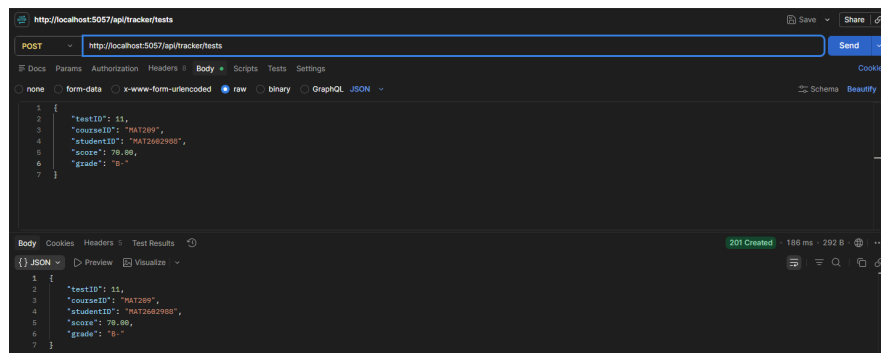


XIAMEN UNIVERSITY MALAYSIA

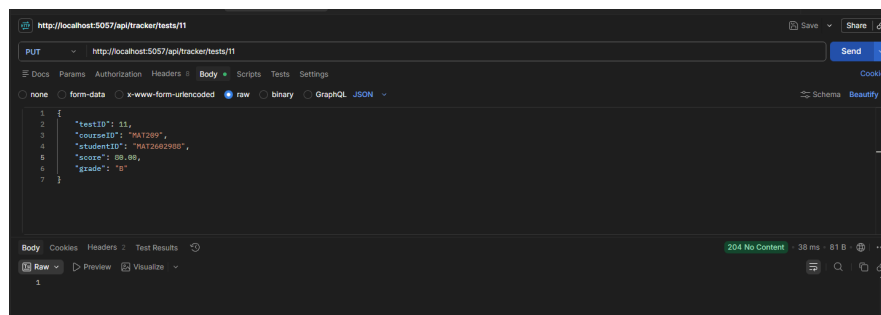
GET: <http://localhost:5057/api/tracker/tests/1>



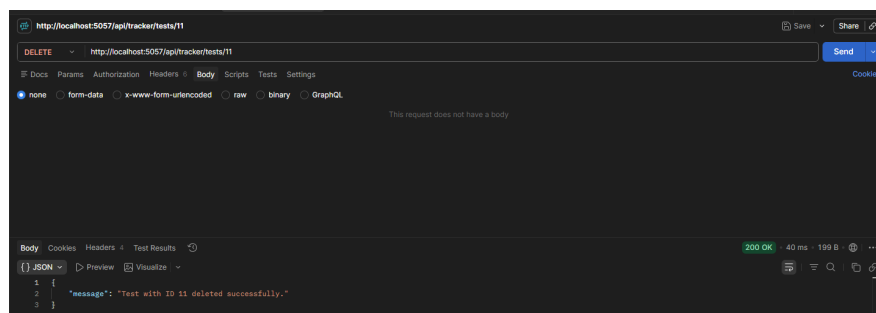
POST : <http://localhost:5057/api/tracker/tests>



PUT : <http://localhost:5057/api/tracker/tests/11>

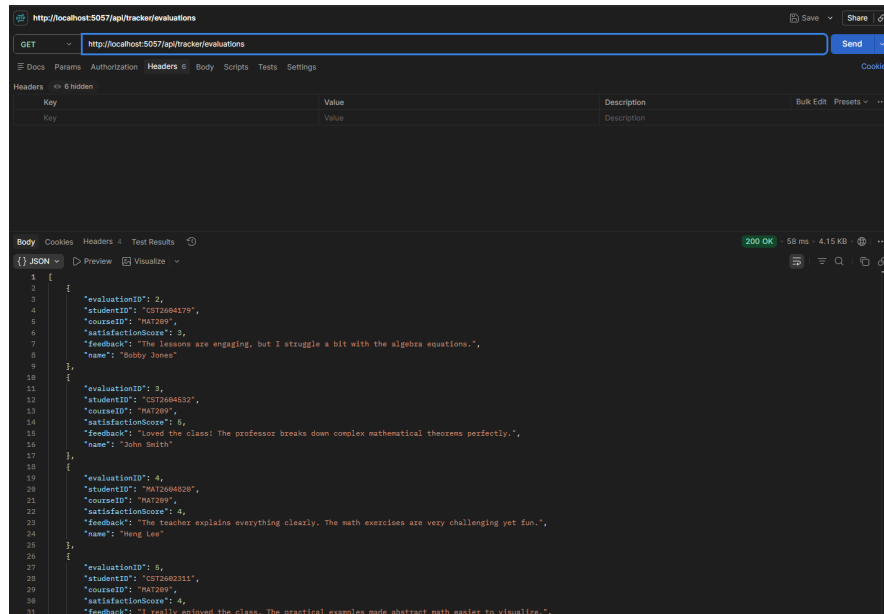


DELETE ; <http://localhost:5057/api/tracker/tests/11>

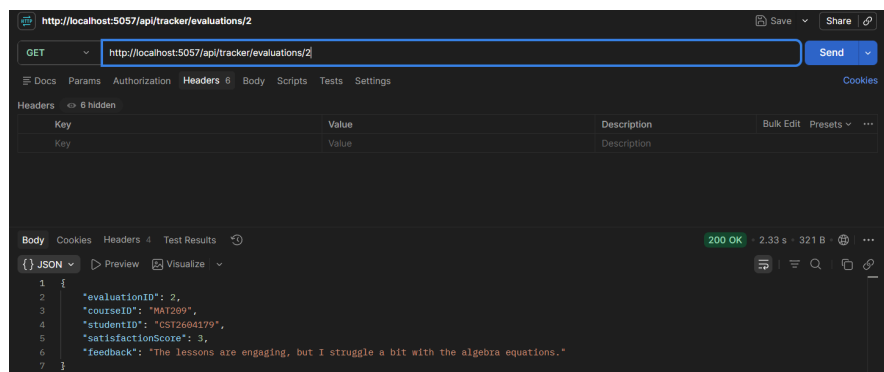


3.3 Student Evaluation Table

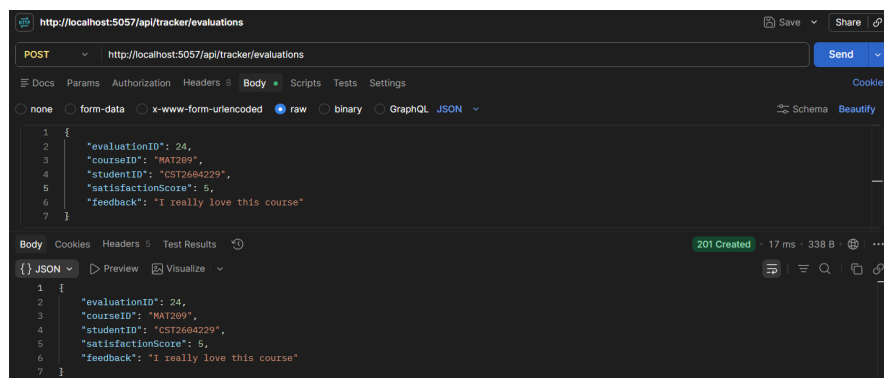
GET : <http://localhost:5057/api/tracker/evaluations>



GET : <http://localhost:5057/api/tracker/evaluations/2>

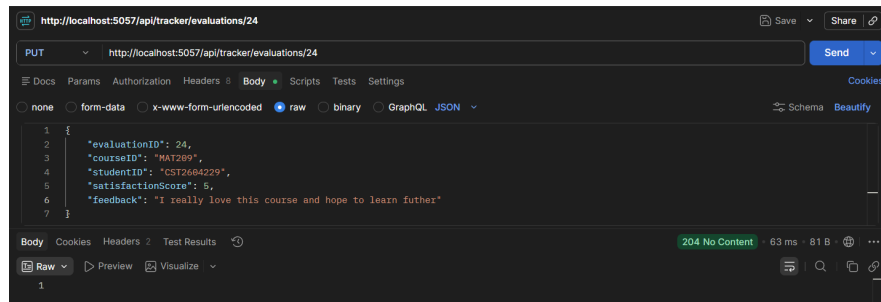


POST : <http://localhost:5057/api/tracker/evaluations>

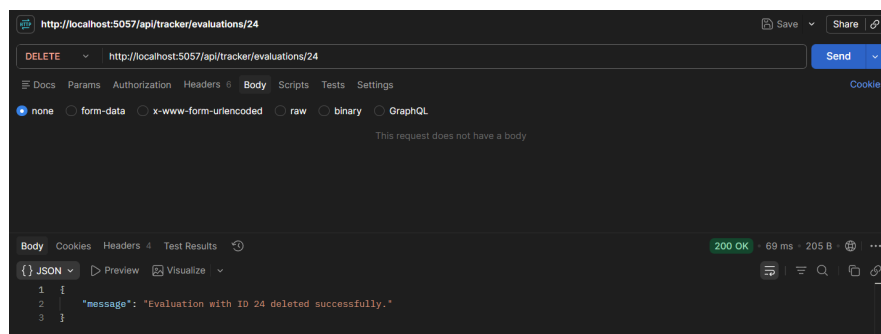


XIAMEN UNIVERSITY MALAYSIA

PUT : <http://localhost:5057/api/tracker/evaluations/24>



DELETE : <http://localhost:5057/api/tracker/evaluations/24>



4. UI Design

4.1 DashBoard Page

This is the dashboard page when the teacher first accesses the website. On the top displaying a navigation bar, which can be clicked based on decision. Once one of the navigation buttons is selected, the box button will turn into light blue. Below the navigation bar showing the student performance analysis that represents in a form of chart. The first chart shows the frequency of students getting a score between 0 to 100 for an assignment and the second chart shows the frequency of students getting a score between 0 to 100 for a test.

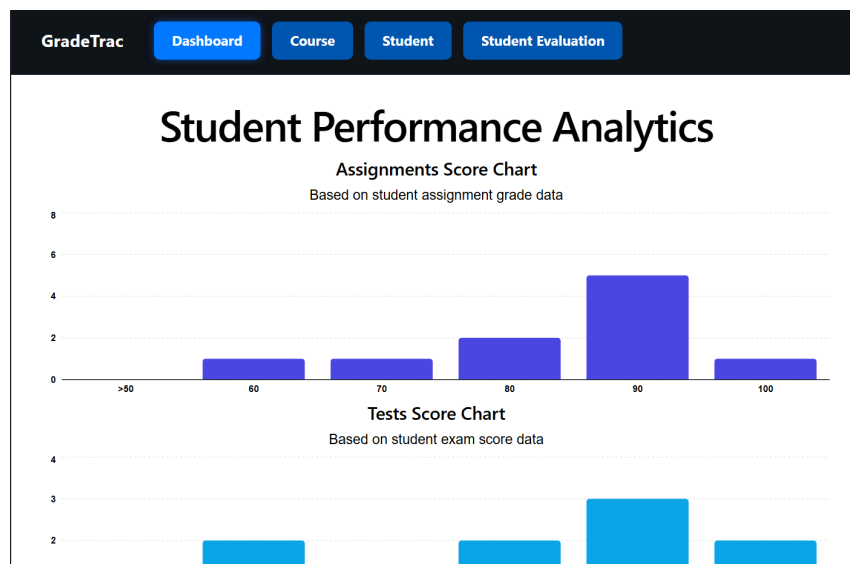
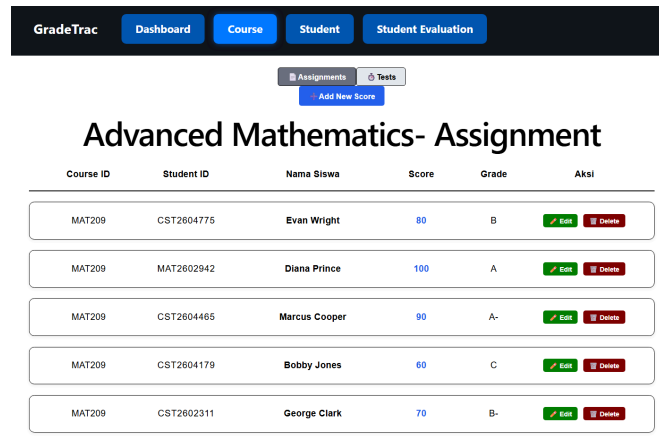


Figure 4.1 Dashboard Page

4.2 Course Page

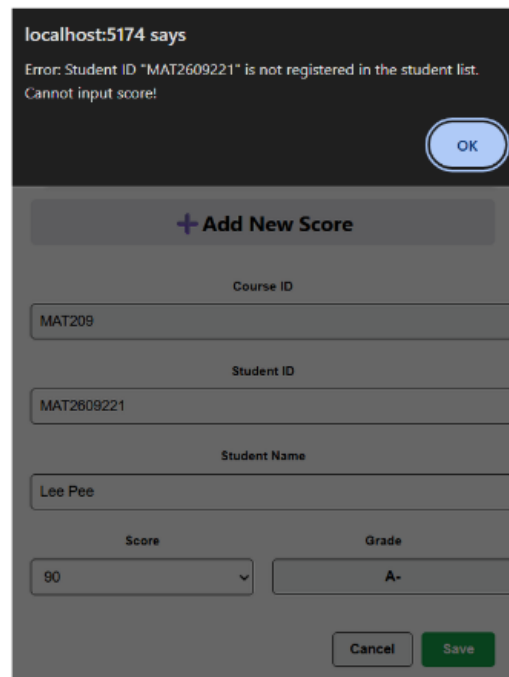
Move to the next page, which is Course Page. In this page there are two options (assignment/test) the teacher is able to select between one of them, each of the options showing a list of student scores. Teachers are able to add a new score input, edit, dan deleting the existing data.



Course ID	Student ID	Nama Siswa	Score	Grade	Aksi
MAT209	CST2604775	Evan Wright	80	B	Edit Delete
MAT209	MAT2602942	Diana Prince	100	A	Edit Delete
MAT209	CST2604465	Marcus Cooper	90	A-	Edit Delete
MAT209	CST2604179	Bobby Jones	60	C	Edit Delete
MAT209	CST2602311	George Clark	70	B-	Edit Delete

Figure 4.2.1 Course Page

In order for the teacher to add a new score either assignment or test, the student should be registered in the student list. If student id / name is not listed in the student list the system will display an error message when the teacher submits the score.



localhost:5174 says

Error: Student ID "MAT2609221" is not registered in the student list.
Cannot input score!

OK

+ Add New Score

Course ID
MAT209

Student ID
MAT2609221

Student Name
Lee Pee

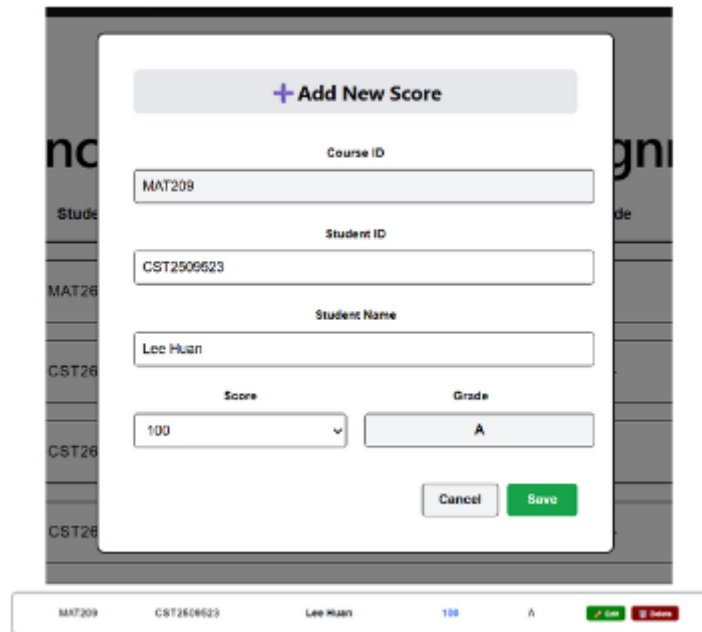
Score
90

Grade
A-

Cancel Save

Figure 4.2.2 Error Proof

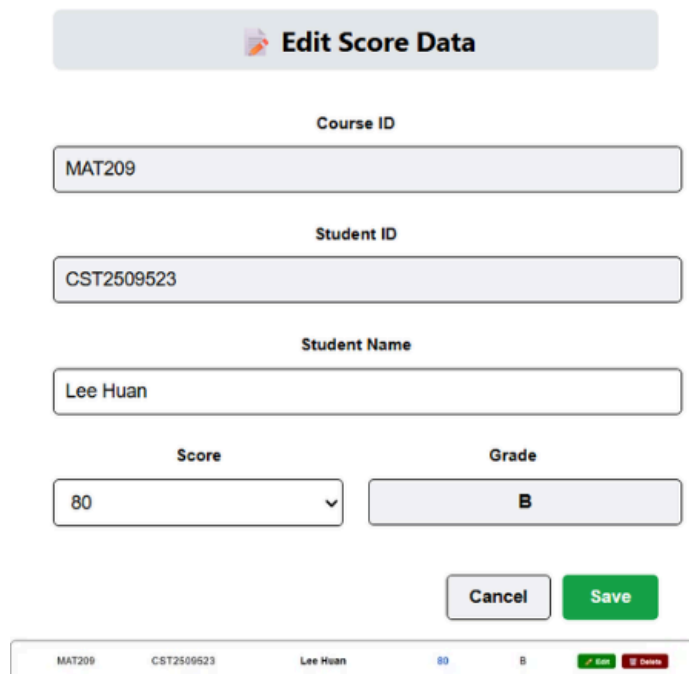
If the student ID/name in the student list, teacher can key in the score and displaying a successful update on the Assignment Score List



Course ID	Student ID	Student Name	Score	Grade	Actions
MAT209	CST2509523	Lee Huan	100	A	Edit Delete

Figure 4.2.3 Add New Score

Teachers are also able to edit existing scores or delete existing scores in the list. The score list will update once the process is done.



Course ID	Student ID	Student Name	Score	Grade	Actions
MAT209	CST2509523	Lee Huan	80	B	Edit Delete

Figure 4.2.3 Edit Existing Score

4.3 Student Page

This page is displaying all students who have been successfully enrolled in the course. Each student includes some important information such as name, studentID, year of intake, how old is the student and email address. There several button which allow lecturer or teacher to add new student, edit student information or even delete existing student

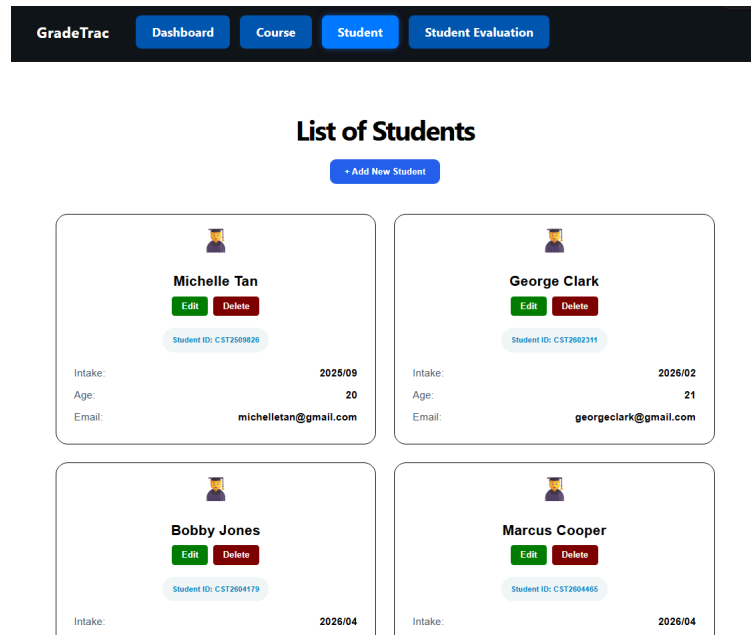


Figure 4.3.1 Student Page

When a new student is coming, the teacher should register the student by “Add New Student” and it will pop up a page that should be filled in each column based on the student information. Once the teacher has the key in all the required information and clicks ‘Save’, new students that have been registered will be displayed in the student list page.

The image shows two parts of the GradeTrac interface. The top part is a screenshot of the 'Add Student' modal form. The form has a title 'Add Student' and several input fields: 'MAT2609111' (likely a course or ID), 'Leo Wang' (name), '2026/09' (intake date), '20' (age), and 'leo.wang@gmail.com' (email). There are 'Save' and 'Cancel' buttons at the bottom of the form. The background shows a list of students with entries for Michelle Tan and George Clark.

The bottom part is a detailed view of the student card for 'Leo Wang'. It features a student icon, the name 'Leo Wang', and 'Edit' and 'Delete' buttons. Below the name is the 'Student ID: MAT2609111'. At the bottom, the student's details are listed: 'Intake: 2026/09', 'Age: 20', and 'Email: leo.wang@gmail.com'.

Figure 4.3.2 Student Evaluation - Add New Student

Teachers are able to edit students information by clicking the green button “Edit”, once the edit process has been done, the teacher can select save. The new student information is updated in the list. Beside that, "Delete" button that represents in red color is able to delete existing student information. Once delete action is executed, the data in the student list is gone.

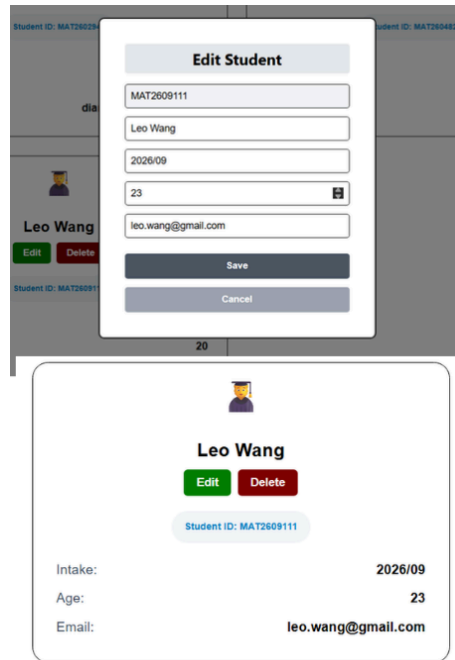


Figure 4.3.3 Student Evaluation - Edit Student Information

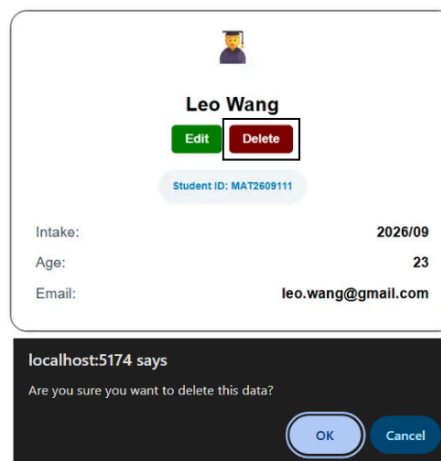


Figure 4.3.4 Student Evaluation - Delete Existing Student

4.4 Student Evaluation Page

In this page, the teacher or lecturer is able to see student evaluation for the course. List of students with detailed information such as studentID, name, course taken, satisfaction score toward the course and feedback/comment about course for compliment or improvement.

GradeTrac				
Dashboard Course Student Student Evaluation				
Student Evaluation				
List of student satisfaction and feedback results for the course.				
Student ID	Name	Course Code	Satisfaction Rate	Feedback
CST2604179	Bobby Jones	MAT209	3 / 5 ★	"The lessons are engaging, but I struggle a bit with the algebra equations."
CST2604532	John Smith	MAT209	5 / 5 ★	"Loved the class! The professor breaks down complex mathematical theorems perfectly."
MAT2604820	Heng Lee	MAT209	4 / 5 ★	"The teacher explains everything clearly. The math exercises are very challenging yet fun."
				"I really enjoyed the class. The practical

Figure 4.4 Student Evaluation Page

Github Link for Front End & Back End Code:

https://github.com/williamlaurenz/SWE310_Assignment

APPENDIX 1

Marking Rubrics

Criteria	Excellent	Good	Average	Needs Improvement	Weight (%)	Marks
Part 1 (Database Design)	Complete ERD, normalized schema (3NF), all constraints and relationships, comprehensive seed data.	Good schema design with most constraints present, seed data included.	Basic tables present, some constraints missing, minimal seed data.	Incomplete or poorly designed schema, missing constraints or seed data.	10	
Part 2 (API – Endpoints)	All CRUD endpoints for 3+ entities, correct HTTP codes, proper JSON responses, no errors.	Most endpoints work correctly with minor issues.	Partial CRUD implementation; some endpoints are missing or returning incorrect responses.	Few or no working endpoints.	20	
Part 2 (API – Validation & Error Handling)	Comprehensive input validation and global error handling with meaningful messages throughout.	Validation is present on most inputs, and error handling covers common cases.	Basic validation only; error handling is inconsistent or incomplete.	Little to no validation or error handling implemented.	10	
Part 3 (Frontend – CRUD Views)	All CRUD views implemented and functional for all entities; smooth API integration with no errors.	Most views functional; minor issues in one or two areas.	Some views implemented; notable gaps in CRUD functionality.	Few or no functional CRUD views.	25	
Part 3 (Frontend – UI & Validation)	Clean, consistent, and responsive UI; form validation and error/loading states all handled well.	Good UI with styling applied; validation mostly present.	Basic styling and limited form validation.	Poor or missing UI styling and form validation.	15	
Part 4 (Code Quality)	Consistent naming, modular structure, well-commented, no hardcoded values, clean and readable.	Generally clean code with minor inconsistencies.	Some structure, but inconsistent naming or comments.	Disorganized code, poor naming, and no comments.	10	
Report Writing	Well-structured, clear writing; covers all	Clear and mostly complete report	The report is present, but some sections	Report largely incomplete or incoherent.	10	

	required sections with relevant screenshots.	with minor omissions.	are missing or unclear.			
TOTAL					100	

Note to students: Please include the marking rubric when submitting your coursework.